

Helix OTA Server — Health Verify + Harden Remediation

Revision: 1 Last modified: 2026-07-09T00:00:00Z

Companion to REPORT.md (the prior baseline audit). Every command below was actually executed in /home/milos/Factory/projects/tools_and_research/helix_ota/server (module github.com/HelixDevelopment/helix_ota/server); every exit code shown is verbatim. No source files were modified. No git add/commit/push was performed (conductor serializes per guardrails). Anti-bluff (§11.4): all verdicts carry captured command output; no cause is guessed (§11.4.6).

Scope

Re-verify the four mandated project checks against the current working tree, systematic-debug + fix any FAILURE / non-empty gofmt list / vet warning (§11.4.102 — no fix without a proven root cause), and record final GREEN evidence.

Toolchain

go version go1.26.4-X:nodwarf5 linux/amd64

Check results (this run)

#	Check	Command (from server/)	Result	Exit
1	Build	go build ./...	GREEN — no compiler output	0
2	Vet	go vet ./...		0

#	Check	Command (from server/)	Result	Exit
			GREEN — no diagnostics	
3	Format	<code>gofmt -l .</code>	GREEN — zero files listed	0
4	Tests	<code>go test -count=1 ./...</code>	GREEN — 15/15 tested pkgs ok, 1 no-test-files, exit 0	0

1. `go build ./...`

`BUILD_EXIT=0`

No compiler output preceded the exit line — clean build.

2. `go vet ./...`

`VET_EXIT=0`

No vet diagnostics — clean.

3. `gofmt -l .`

(no files listed)

`GOFMT_LIST_ABOVE_EXIT=0`

`gofmt -l` prints a path per unformatted file. Zero paths were printed → tree is fully `gofmt`-clean. (Note: `gofmt -l` itself always exits 0; the empty file list is the signal.)

4. `go test -count=1 ./... (forced, no cache)`

A first `go test ./...` returned mostly (cached). Per §11.4.6/§11.4.5 a cached result is not fresh captured evidence, so the run was repeated with `-count=1` to force genuine re-execution. Verbatim:

```
ok      .../cmd/applyport    0.928s
ok      .../cmd/ota-device-emu 0.925s
ok      .../cmd/ota-server   2.439s
ok      .../internal/api      0.264s
?      .../internal/api/manager-dist [no test files]
ok      .../internal/config 0.002s
ok      .../internal/device 0.183s
ok      .../internal/deviceemu 0.227s
ok      .../internal/fabric 0.004s
ok      .../internal/health 0.002s
ok      .../internal/rollout    0.003s
ok      .../internal/store    0.004s
ok      .../internal/transport 0.111s
ok      .../tests/chaos 0.024s
ok      .../tests/stress    0.014s
ok      .../tools/loadtest 1.604s
TEST_COUNT1_EXIT=0
```

15/15 tested packages PASS, 0 FAIL, 1 package (internal/api/manager-dist) has no `_test.go` files. Slowest package cmd/ota-server at 2.439s — no hang, no timeout, well under the 400s budget.

Fixes applied

None required. All four mandated checks were already GREEN on the current working tree. There was no FAILURE, no non-empty gofmt list, and no go vet warning to root-cause or repair. Per the anti-gold-plating discipline, no source change was invented where no defect exists (§11.4.1 — a needless change is a new failure-mode risk; §11.4.124 — do not touch working code without a proven reason). The §11.4.102 systematic-debugging arc therefore did not fire: its Iron Law (“no fix without a root cause”) means the correct action for a zero-finding tree is to change nothing.

Honesty note — topology-gated integration tests (not a failure)

The default `go test ./...` does not exercise the Postgres integration suite. 11 test files under `internal/store/` and `internal/rollout/` are gated behind `//go:build integration` and require a live `pgx/`

PostgreSQL instance (§11.4.3 topology-dependent). This is correct SKIP-by-absence, not a hidden failure. Verified they are not rotting behind the tag: they compile + vet cleanly under the tag —

```
go vet -tags integration ./internal/store/ ./internal/rollout/  
VET_INTEG_EXIT=0
```

Gated files (evidence, for the tracker): internal/{store,rollout}/postgres_integration_test.go, postgres_coverage_integration_test.go, postgres_fault_integration_test.go, pg_itest_lock_test.go, faultproxy_test.go, plus internal/store/postgres_fabric_integration_test.go. Running them requires -tags integration + a Postgres endpoint; they are out of scope for the four default-invocation checks and remain a topology-gated suite, not a defect.

Coverage gap carried forward (from REPORT.md — not a failure)

internal/api/manager-dist has zero _test.go files. Flagged here as an honest tracked coverage gap (§11.4.15-class item), not a build/test failure. Unchanged from the baseline report.

Verdict

Build / vet / gofmt / test: **GREEN** → **GREEN** (no regression, no fix needed). Nothing remains unresolved within the four mandated checks. The Postgres integration suite and the manager-dist coverage gap are pre-existing, honestly-tracked items outside the default-invocation scope — not blockers.